

Rapport Détaillé : Algorithmes de Flot Maximal et Coût Minimal

Achour DJERADA

6 mai 2026

1 Introduction et Objectif du Projet

Ce projet implémente une bibliothèque complète de résolution de problèmes classiques de la recherche opérationnelle et de l'optimisation dans les graphes :

- **Le problème du flot maximal** (faire transiter un maximum d'unités de la source au puits).
- **Le problème du flot de coût minimal** (envoyer ce flot selon les chemins les moins chers).
- **La détection de cycles de coût négatif** (qui risqueraient de faire diverger les solveurs de coût minimal).

Pour y parvenir de manière optimale, tout le socle du projet repose sur une structure de **graphe résiduel**.

2 Structure de Données : Le Graphe Résiduel (src/graph.py)

C'est la pièce maîtresse du projet. Dans les algorithmes de flot, on ne se contente pas de traverser les arcs existants : il faut pouvoir **annuler ou rediriger un flot déjà envoyé**. C'est le rôle des arcs résiduels.

2.1 L'objet Edge

Chaque arc est défini ainsi grâce aux **dataclasses** de Python :

```
@dataclass
class Edge:
    to: int          # Sommet de destination
    rev: int          # Indice de l'arc inverse dans la liste de destination
    cap: int          # Capacité (résiduelle) de l'arc
    cost: int         # Coût unitaire de l'arc
    original_cap: int # Sauvegarde de la capacité pour générer la sortie
    is_reverse: bool  # Différencie les arcs d'origine des virtuels
```

2.2 Pourquoi cette implémentation ? Les listes d'adjacence couplées (rev)

Le graphe conserve ses arcs dans une liste de listes `list[list[Edge]]` (`adjacency[u]`).

1. **Listes d'adjacence vs Matrice d'adjacence** : Les graphes réseaux sont souvent creux ($E \ll V^2$). Les listes d'adjacence permettent d'itérer sur les voisins en un temps proportionnel au degré du sommet $\mathcal{O}(\deg(u))$ au lieu d'un parcours linéaire lent en $\mathcal{O}(V)$ de la matrice.

2. **L'astuce de l'indice rev** : Quand on crée un arc de $u \rightarrow v$ avec une capacité C , on crée silencieusement un arc inverse de $v \rightarrow u$ de capacité 0 et de coût $-cost$. L'attribut **rev** stocke l'indice de cet arc jumeau. **Avantage immédiat en complexité** : Lorsqu'un algorithme pousse Δ flot sur $u \rightarrow v$, il réduit sa capacité de Δ et augmente la capacité de l'arc inverse de Δ en $\mathcal{O}(1)$ (`reverse.cap += pushed_flow`). Sans cet attribut **rev**, il faudrait chercher l'arc inverse dans la liste des voisins de v en $\mathcal{O}(\deg(v))$.

3 Algorithmes et Complexités Théoriques

3.1 Flot Maximal : Algorithme d'Edmonds-Karp (`src/max_flow.py`)

La fonction `ford_fulkerson_max_flow` porte le nom académique générique, mais elle implémente en réalité **Edmonds-Karp** car elle utilise une recherche en largeur (BFS) :

- **Principe** : La fonction `find_augmenting_path` effectue un parcours en largeur (BFS) depuis la source pour trouver le chemin le plus court (en nombre d'arcs, sans tenir compte du coût) jusqu'au puits, avec des arcs ayant une capacité résiduelle > 0 .
- **Mise à jour** : `apply_augmenting_path` met à jour les capacités le long du chemin en $\mathcal{O}(V)$.
- **Complexité** : Chaque BFS prend $\mathcal{O}(E)$. Dans un réseau, le nombre maximal d'augmentations avec des chemins les plus courts est borné par $\mathcal{O}(V \cdot E)$.
- **Complexité finale** : $\mathcal{O}(V \cdot E^2)$ ce qui assure l'arrêt rapide même sur des valeurs de flots irrationnelles ou très grandes.

3.2 Flot de Coût Minimal (`src/min_cost_flow.py`)

Ce problème est résolu par la méthode des *chemins les plus courts successifs* (Successive Shortest Path algorithm).

A. Avec Bellman-Ford

- **Principe** : Dans le graphe résiduel, la création des arcs inverses génère des coûts négatifs. L'algorithme de Dijkstra classique ne peut donc pas être utilisé directement. La fonction `min_cost_max_flow_bellman_ford` lance donc une recherche de plus court chemin via **Bellman-Ford** à chaque itération.
- **Complexité** : Le calcul de Bellman-Ford coûte $\mathcal{O}(V \cdot E)$. S'il y a F unités de flot total à envoyer, la méthode est pseudo-polynomiale et tourne en $\mathcal{O}(F \cdot V \cdot E)$.

B. Avec Dijkstra et ajustement par Potentiels (Algorithme de Johnson)

- **Principe** : Pour contourner la lenteur de Bellman-Ford, la fonction `shortest_path_dijkstra_with_pots` utilise un système de prix (les potentiels π). On exécute Bellman-Ford **une seule fois** pour trouver les distances initiales de la source $\pi(v)$. On modifie mathématiquement tous les coûts : $c'(u, v) = c(u, v) + \pi(u) - \pi(v) \geq 0$. Tous les coûts modifiés étant dorénavant positifs, on peut enchaîner les recherches de chemins via **Dijkstra**.
- **Complexité de Dijkstra avec tas minimum** : $\mathcal{O}(E \log V)$.
- **Complexité finale** : $\mathcal{O}(V \cdot E)$ (initialisation) + F itérations de Dijkstra, donnant $\mathcal{O}(V \cdot E + F \cdot E \log V)$.

3.3 Détection de Cycle Négatif (`src/cycle_detection.py`)

- **Principe** : La fonction `has_negative_cycle` simule la création d'un "Super Nœud" lié à tous les nœuds par des arêtes de coût 0 (soit $dist = 0$ pour tous). S'il y a une relaxation

- valide possible après exactement $V - 1$ boucles de relaxation, alors le graphe contient un cycle de poids négatif.
- **Complexité** : $\mathcal{O}(V \cdot E)$.

4 Conclusion

L'architecture de l'application est pensée pour éviter des duplications de code et limiter l'empreinte mémoire, centralisée autour de ce graphe résiduel partagé. La pertinence de la structure de données `Edge` par référence croisée indexée permet de tenir des temps de calcul théoriques respectables tout en assurant l'extensibilité du code.

5 Bibliographie

1. **Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993).** *Network Flows : Theory, Algorithms, and Applications*. Prentice Hall.
2. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
3. **Tarjan, R. E. (1983).** *Data Structures and Network Algorithms*. SIAM.